MaRENet: Masked Attention and Recurrent Encoding for Transaction-Level Credit Card Fraud Detection

Duc-Anh Nguyen, Kim-Ngan Thi Nguyen, Trong-Nghia Nguyen, Vu-Duc Ngo
Huan Vu, Tien-Cuong Nguyen, Mai Xuan Trang, and Thien Van Luong

***Abstract*—Detecting credit card fraud remains challenging because fraudulent events are rare, are embedded in long streams of legitimate activity, and depend on subtle temporal cues. We introduce MaRENet (Masked Attention and Recurrent Encoding Network), which represents each transaction with a fixed-length history window and strengthens day-of-week and hour-of-day signals through Temporal Feature Scaling. MaRENet then applies Channel-wise Feature Attention to assign higher weights to more informative transaction features. This helps the recurrent encoder focus on features that are more useful for fraud detection. MaRENet is trained using binary cross-entropy and is evaluated on two large-scale public transaction datasets. It achieves F_1-scores of 0.9694 on Dataset 1 and 0.9883 on Dataset 2, outperforming all compared baselines. Additional experiments with a time-ordered split show that MaRENet remains effective when trained on earlier transactions and tested on later ones. In addition, under the reported GPU inference benchmark, MaRENet-GRU achieves lower latency and higher throughput than TH-LSTM while using fewer trainable parameters.**

***Index Terms*—Fraud detection, recurrent neural networks, attention mechanisms, temporal scaling, class imbalance.**

I. Introduction

Credit card fraud is widespread and costly but remains difficult to detect because it is rare. Attackers use methods such as Trojan malware and credential stuffing to steal money [1], leading to global losses of about $35 billion in 2020 [2]. Fraudulent transactions usually make up less than 1% of all transactions, so they are easily hidden among normal ones and often bias models toward the majority class [3], [4], [5], [6]. Another challenge is that user behavior changes over time and across locations. Fraudsters often act in short active periods or specific regions, while genuine users show stable and periodic spending patterns. These factors make fraud detection both an imbalanced and dynamic learning problem.

Recent studies have moved from traditional classifiers to sequential models that learn user behavior over time. Recurrent neural networks are widely used to capture temporal patterns in transaction sequences. For example, TH-LSTM [7] uses time gaps between transactions to update its memory for irregular activity, while STGN [8] combines temporal and geographic signals to detect location-related fraud. Feature quality is also important: SEDA [9] automatically selects useful feature subsets to reduce noise and improve performance without manual feature engineering. Despite these advances, many methods do not explicitly reweight periodic temporal cues and lack a lightweight mechanism for highlighting informative features, which motivates our MaRENet.

In this work, we propose MaRENet, a neural framework for credit card fraud detection that combines Transactional Expansion, Temporal Feature Scaling, Channel-wise Feature Attention, and a GRU/LSTM recurrent encoder. Each component has a distinct role: Transactional Expansion builds the recent transaction history of the cardholder, Temporal Feature Scaling rescales only selected temporal features (day-of-week and hour-of-day), Channel-wise Feature Attention then reweights all feature channels with a learnable mask, and the recurrent encoder learns temporal dependencies across the resulting sequence. In this way, MaRENet combines recent history, temporal cues, channel-wise feature weighting, and recurrent sequence modeling within a single framework. Our main contributions are:

- **Temporal and contextual modeling:** MaRENet represents each transaction with a fixed-length history window and applies *Temporal Feature Scaling* to strengthen day-of-week and hour-of-day signals, improving the detection of regular behavior patterns.
- **Lightweight feature attention:** We introduce a *Channel-wise Feature Attention* block, inspired by the channel-masking idea in Shift-GCN [10] but used without graph computation, to reweight input features and highlight informative channels with low overhead.
- **Learning under class imbalance:** By combining recent history with temporal scaling and channel-wise masking, MaRENet better captures minority-class cues and remains stable under highly imbalanced data.
- **Extensive evaluation and efficiency:** We evaluate MaRENet on two large-scale public transaction datasets with 1.85M transactions from 999 cardholders (0.52% fraud) and 8.58M transactions from 9,968 users (1.10% fraud). MaRENet achieves F_1-scores of 0.9694 and 0.9883. We further evaluate MaRENet with a time-

Duc-Anh Nguyen, Huan Vu, Trong-Nghia Nguyen, Kim-Ngan Thi Nguyen, Thien Van Luong are with Business AI Lab, College of Technology, National Economics University Hanoi, Vietnam (e-mail: anhnd.bai@st.neu.edu.vn, {huanv, nghiant, ngannguyen, thienlv}@neu.edu.vn).

Tien-Cuong Nguyen is with VNPT AI, VNPT Group, Hanoi, Vietnam (e-mail: nguyentiencuongn@vnpt.vn).

Vu-Duc Ngo is with MobiFone HighTech Center, MobiFone Corp., Hanoi, Vietnam (e-mail: duc.ngo@mobifone.vn).

Mai Xuan Trang is with A2I Lab, School of Computing, Phenikaa University, Hanoi, Vietnam (e-mail: trang.maixuan@phenikaa-uni.edu.vn).

Corresponding author: Thien Van Luong (e-mail: thienlv@neu.edu.vn).

ordered split, report imbalance-aware metrics (AUPRC and Recall@FPR$\leq 1\%$), and measure inference latency, throughput, and memory usage under a fixed benchmark setting.

II. Related Work

Credit card fraud detection has been studied for many years using both traditional machine learning (ML) and deep learning (DL) methods. Earlier work mainly used supervised classifiers that treated each transaction as an independent record. More recent studies focus on sequential models that learn how user behavior evolves over time.

Traditional ML methods often depend on hand-crafted features and ensemble techniques to capture summary patterns. In contrast, DL approaches model the transaction stream as a time series and use recurrent or attention-based networks to learn behavioral patterns automatically.

A. Machine Learning for Fraud Detection

Traditional machine learning methods are common baselines for credit card fraud detection. Early studies used distance-based and linear models such as K-Nearest Neighbors (KNN) and Logistic Regression. KNN flags fraud by comparing each transaction with similar past samples and is often paired with dimensionality reduction [11]. Logistic Regression remains popular due to its simplicity and interpretability, and recent work improves it via feature selection, SMOTE balancing, and hyperparameter tuning [12].

Later, more expressive models based on decision trees were widely adopted. Single decision trees are attractive, and they have been used for real-time monitoring of transaction behavior [13]. However, they often overfit and cannot fully cover the wide range of fraud patterns. To address these issues, tree-based ensemble methods such as Random Forests, LightGBM, XGBoost, and CatBoost have become strong choices for structured transaction data. Random Forests improve robustness by combining many trees [14], while gradient boosting frameworks such as LightGBM [15], XGBoost [16], and CatBoost [17] achieve high performance on large and imbalanced datasets.

Ensemble strategies such as stacking and voting further combine different classifiers. Stacking trains a meta-model on the outputs of base learners [18], while voting aggregates predictions (e.g., by averaging probabilities) to improve stability [19].

Overall, ML and ensemble models can perform well, but they have clear limits for fraud detection. Many treat each transaction independently, which ignores sequential user behavior and contextual signals. In addition, even with resampling, they remain sensitive to severe class imbalance and may degrade under distribution drift over time.

B. Deep Learning Models

In recent years, deep learning models such as recurrent neural networks (RNN) have been widely used for fraud detection. For instance, [20] proposed a bidirectional GRU architecture that improves the modeling of transaction sequences and helps detect subtle fraud behaviors more effectively. However, the GRU still treats all input features in the same way and does not explicitly focus on which temporal cues are most important for the prediction. In related work, long short-term memory (LSTM) networks have also been applied to fraud detection. This line of research models the task as a sequence classification problem, where each transaction is processed together with its recent history rather than in isolation [21]. This approach brings clear benefits, since the model can capture temporal patterns in user behavior. However, LSTM models are often more computationally demanding and, like GRUs, still treat individual feature channels in a uniform way, which makes it hard to highlight the most important temporal cues.

Beyond these baseline RNN models, several more advanced architectures have been proposed for credit card fraud detection. Time-aware historical-attention-based LSTM (TH-LSTM) is one such model [7]. It extends the standard LSTM with a time-aware gate that uses the time gaps between transactions, a current–historical attention module that focuses on the most relevant past steps, and an interaction module that combines the original features, hidden states, and attention context. In this way, TH-LSTM learns stronger behavioral representations and can better separate normal and fraudulent transaction sequences. Another recent model is the Spatio-Temporal Attention Network (STAN) [22]. STAN combines spatio-temporal attention with 3D convolutional networks for fraud detection. It first builds temporal and spatial feature slices from transaction records, and then uses attention layers to learn which time windows and locations are most important. After that, a 3D convolutional network learns local patterns over time, space, and feature dimensions, which helps the model capture complex fraud behaviors.

Overall, deep learning models have clear benefits for credit card fraud detection. GRU and LSTM can read transactions as sequences and learn how user behavior changes over time, which already improves detection compared with single-transaction models. More advanced models such as TH-LSTM and STAN go further by using time-aware gates, attention, and spatio-temporal structures to capture complex patterns. However, these models are often complex and computationally heavy; they still require some manual feature design, and their computational behavior is not always examined under long transaction histories.

III. Proposed Method

The overall workflow of MaRENet is shown in Fig. 1. MaRENet performs transaction-level fraud detection: for each current transaction, the model uses the user's recent transaction history to predict whether that transaction is fraudulent or legitimate. The pipeline starts from raw transaction data, which are cleaned, encoded, and normalized in the data preprocessing step. Each transaction is then combined with its most recent past transactions to form a fixed-length window. Next, *Temporal Feature Scaling* increases the influence of the day-of-week and hour-of-day features, and a *Channel-wise Feature*

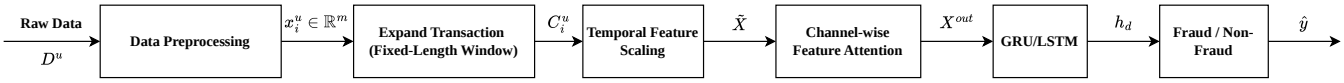


Fig. 1. Overall pipeline of the proposed MaRENet model.

Attention block reweights input channels to emphasize informative signals. The window is then encoded by an RNN (GRU or LSTM) to model temporal dependencies, and a sigmoid layer outputs the probability that the current transaction is fraudulent.

A. Data Preprocessing

To prepare the tabular transaction data for MaRENet's sequential modeling, duplicate records are removed, basic consistency checks are performed, and transaction timestamps are parsed. From the parsed time fields, simple periodic signals are derived, namely *day-of-week* and *hour-of-day*.

The dataset is split before preprocessing. Categorical features are encoded without using the fraud label. Binary features and categorical features with a small number of values are one-hot encoded, while the remaining categorical features are integer encoded. All categorical encoders and numerical scalers are fitted using the training split only. Categorical feature values appearing in test transactions but not observed in the training data are assigned a reserved unknown value. Day-of-week and hour-of-day are kept as numeric features for Temporal Feature Scaling. Raw fields that are not used directly as model inputs, such as `first`, `lat`, `long`, `merch_lat`, `merch_long`, and `trans_date_trans_time`, are removed after the required derived features are created.

B. Transactional Expansion

In real financial systems, looking at a single transaction is usually not enough to understand a user's behavior or detect fraud. A transaction becomes more meaningful when we see it together with the user's recent history. To capture this context, we follow the transactional expansion strategy from TH-LSTM [7]. Each transaction is grouped with a fixed number of previous transactions, creating a short history window with the same length for all samples.

The workflow is visually summarized in Fig. 2. The diagram traces how raw transaction vectors for a user are standardized and expanded into fixed-length windows. Formally, let $u \in \mathcal{U}$ denote a user, associated with a complete personal sequence of transactions, where $\mathcal{U}$ is the set of all users:

$$\mathbf{D}_u = \{\mathbf{x}_1^u, \mathbf{x}_2^u, \ldots, \mathbf{x}_{n_u}^u\}, \quad \mathbf{x}_i^u \in \mathbb{R}^m, \tag{1}$$

where $\mathbf{D}_u$ is the full set of transactions made by user u, $\mathbf{x}_i^u$ is the feature vector of the i-th transaction, with $i \in \{1, 2, \ldots, n_u\}$, where n_u denotes the number of transactions of user u. Each vector contains m features, i.e., $\mathbf{x}_i^u = \{x_{i_1}^u, x_{i_2}^u, \ldots, x_{i_m}^u\}$, where m is the number of fields that describe a transaction (for example, amount, time, merchant type, or location).

Following the transactional expansion method from TH-LSTM [7], we give each transaction a simple history by grouping it with its d previous transactions. This creates a fixed-length window of size $d+1$. An example of the expanded windows is illustrated in Fig. 2. For a given memory size d, this expanded context is formally written as:

$$\mathbf{C}_i^u = \left[\mathbf{x}_{i-d}^u, \mathbf{x}_{i-d+1}^u, \ldots, \mathbf{x}_i^u\right] \in \mathbb{R}^{(d+1)\times m}. \tag{2}$$

To ensure each window contains exactly $d+1$ transactions, we apply left-padding using the earliest transaction $\mathbf{x}_1^u$ when $i \leq d$. Specifically, for any $j \in \{0, \ldots, d\}$, if $i - d + j \leq 0$, we set $\mathbf{x}_{i-d+j}^u = \mathbf{x}_1^u$. If the current transaction does not have enough historical data available (e.g., at the start of the user's history), the earliest transaction $\mathbf{x}_1^u$ is duplicated to fill the memory window. This padding ensures uniform window lengths, facilitating easier processing by subsequent neural network components. For instance, consider a user u who has a total of five feature vectors: $\{\mathbf{x}_1^u, \mathbf{x}_2^u, \mathbf{x}_3^u, \mathbf{x}_4^u, \mathbf{x}_5^u\}$. With a memory size of $d = 3$, the resulting expanded windows are illustrated in Fig. 2 and written as:

$$\begin{aligned}
\mathbf{C}_1^u &= [\mathbf{x}_1^u, \mathbf{x}_1^u, \mathbf{x}_1^u, \mathbf{x}_1^u], \\
\mathbf{C}_2^u &= [\mathbf{x}_1^u, \mathbf{x}_1^u, \mathbf{x}_1^u, \mathbf{x}_2^u], \\
\mathbf{C}_3^u &= [\mathbf{x}_1^u, \mathbf{x}_1^u, \mathbf{x}_2^u, \mathbf{x}_3^u], \\
\mathbf{C}_4^u &= [\mathbf{x}_1^u, \mathbf{x}_2^u, \mathbf{x}_3^u, \mathbf{x}_4^u], \\
\mathbf{C}_5^u &= [\mathbf{x}_2^u, \mathbf{x}_3^u, \mathbf{x}_4^u, \mathbf{x}_5^u].
\end{aligned} \tag{3}$$

Each expanded window includes the current transaction and its recent history, allowing the model to capture both short-term and longer-term behavior patterns. Choosing the memory length d is important: a large d may weaken recent fraud signals, while a small d may miss repeated or periodic behaviors. Following [7], we set d using the heuristic:

$$d = \left\lfloor \frac{n}{|\mathcal{U}|} \right\rfloor, \tag{4}$$

where $\lfloor \cdot \rfloor$ denotes the floor function (rounding down to the greatest integer less than or equal to its argument), n is the total number of transactions and $|\mathcal{U}|$ is the number of users. This rule-of-thumb sets d close to the average transaction history per user; the resulting window length is $d+1$ when including the current transaction.

It is worth noting that MaRENet uses a fixed transaction history d, which is selected using the heuristic in Eq. (4). The same value of d is used for all users. Since users may have different transaction histories, selecting the history length separately for each user will be considered in future work.

C. Temporal Feature Scaling

Temporal attributes such as *day of the week* and *hour of the day* exhibit strong correlations with fraudulent activity. As shown in Fig. 3 and 4, fraudulent transactions occur more frequently on Monday, weekends (Saturday–Sunday) and during

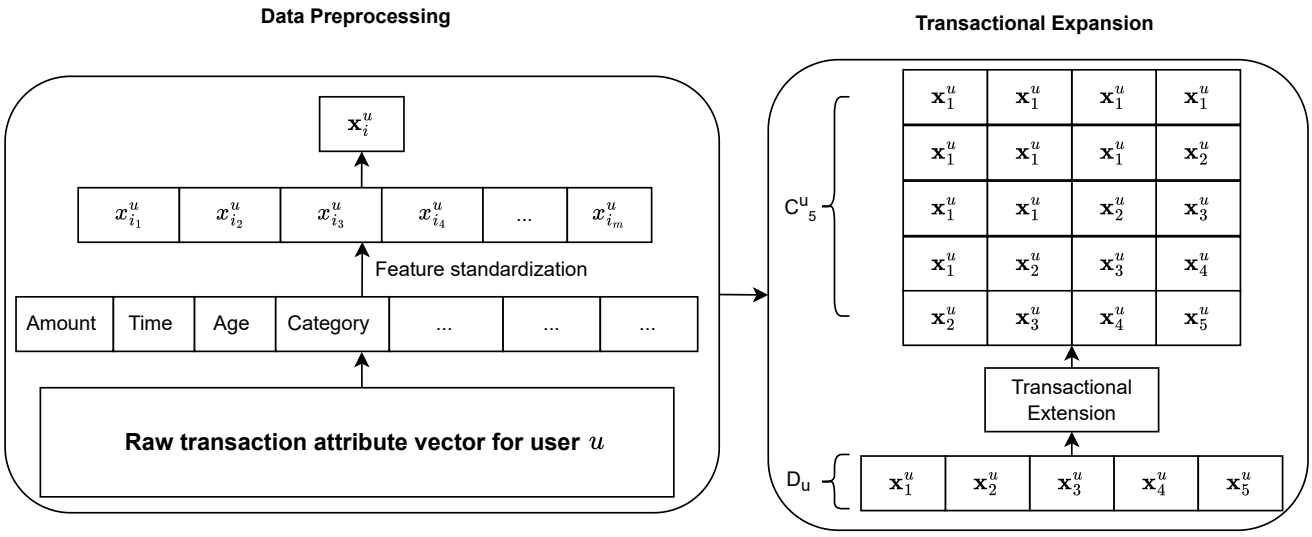


Fig. 2. An example of transactional expansion after data preprocessing (five transactions, $d = 3$).

nighttime hours. These patterns highlight the importance of explicitly modeling temporal cues.

To exploit these signals, we introduce a lightweight feature-modulation mechanism. For notational simplicity, we denote the expanded window $\mathbf{C}_i^u$ obtained in Sec. III-B as the input matrix $\mathbf{X} \in \mathbb{R}^{(d+1)\times m}$, where $d+1$ is the window length and m is the number of feature channels per transaction. Let $\mathrm{idx}_{\mathrm{day}}$ and $\mathrm{idx}_{\mathrm{hr}}$ denote the column indices of the day-of-week and hour-of-day features in $\mathbf{X}$. To incorporate a temporal inductive bias, we assign two trainable scalar weights: w_d for the day-of-week feature and w_h for the hour-of-day feature. Both weights are initialized to 1.0 so that they have a neutral effect at the beginning of training, and they are optimized jointly with the rest of the model through backpropagation. The modulation is applied row-wise to each transaction vector $\mathbf{x}_t$ in $\mathbf{X}$. Let $x_{t,j}$ denote the value of the j-th feature at time step t. The rescaled feature value $\tilde{x}_{t,j}$ is computed as:

$$\tilde{x}_{t,j} = \begin{cases} w_d \cdot x_{t,j}, & \text{if } j = \mathrm{idx}_{\mathrm{day}}, \\ w_h \cdot x_{t,j}, & \text{if } j = \mathrm{idx}_{\mathrm{hr}}, \\ x_{t,j}, & \text{otherwise}, \end{cases} \tag{5}$$

for every time step $t \in \{0, \ldots, d\}$ and every feature index $j \in \{1, \ldots, m\}$. Collecting $\tilde{\mathbf{x}}_t = (\tilde{x}_{t,1}, \ldots, \tilde{x}_{t,m})$ for all t, we obtain the temporally modulated input matrix $\tilde{\mathbf{X}} = [\tilde{\mathbf{x}}_0; \ldots; \tilde{\mathbf{x}}_d] \in \mathbb{R}^{(d+1)\times m}$. Because w_d and w_h are learnable, the model can increase or reduce the day and hour features when they help the fraud prediction.

Temporal Feature Scaling can also be applied to additional temporal features, such as month and the weekend indicator evaluated in our extended experiments. Each added temporal feature has its own learnable parameter. For a selected temporal feature $x_{t,j}$, the scaled value is

$$\widetilde{x}_{t,j} = \Big[1 + \tanh(\alpha_j)\Big] \cdot x_{t,j}, \tag{6}$$

where α_j is learned during training and $s_j = 1 + \tanh(\alpha_j)$ is the effective scaling factor. This bounded multiplicative form follows the mask parameterization used in Shift-GCN [10], but here it is applied only to the selected temporal features.

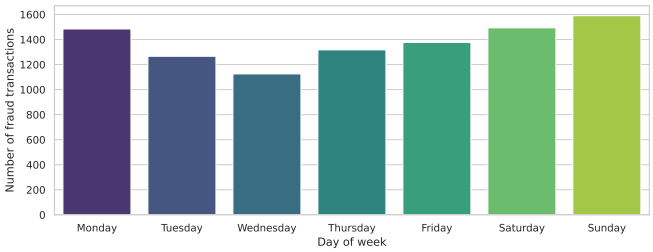


Fig. 3. Distribution of fraud transactions by day of the week.

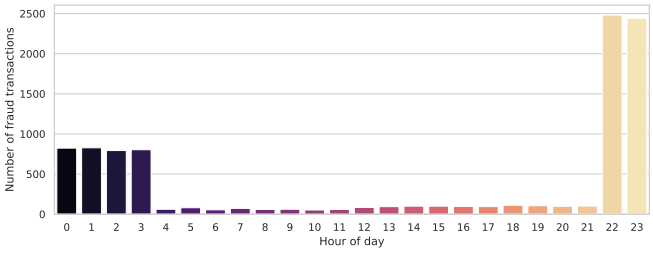


Fig. 4. Distribution of fraud transactions by hour of day.

D. Channel-wise Feature Attention

Following Temporal Feature Scaling, each user window is represented as $\tilde{\mathbf{X}} \in \mathbb{R}^{(d+1)\times m}$. Equation (6) applies scalar scaling only to the selected temporal features. Channel-wise Feature Attention is a separate subsequent operation that

applies a learnable vector mask across all m feature channels. Specifically, we use $\mathbf{m} \in \mathbb{R}^m$, following the mask formulation of Shift-GCN [10]:

$$\mathbf{Z} = \tilde{\mathbf{X}} \odot \Big[1 + \tanh(\mathbf{m})\Big] \in \mathbb{R}^{(d+1)\times m}, \tag{7}$$

where $\odot$ denotes element-wise multiplication. Finally, batch normalization, a residual connection, and a ReLU activation are applied:

$$\mathbf{X}^{\text{out}} = \text{ReLU}\big[\text{Norm}(\mathbf{Z}) + \tilde{\mathbf{X}}\big] \in \mathbb{R}^{(d+1)\times m}. \tag{8}$$

These steps allow the network to emphasize the most informative channels while keeping computation lightweight, thereby improving the quality of the learned representation.

Transactional Expansion, Temporal Feature Scaling, and Channel-wise Feature Attention have different roles before recurrent encoding. Transactional Expansion constructs the recent transaction history. Temporal Feature Scaling modifies only the selected temporal features, such as day-of-week and hour-of-day, through feature-specific scaling factors. Channel-wise Feature Attention is applied afterward to the complete feature representation and uses a learnable channel-wise mask to adjust the contribution of all feature channels. The resulting transaction sequence is then processed by the GRU/LSTM recurrent encoder, as described in Subsection III-E.

MaRENet also differs from temporal self-attention [23]. Temporal self-attention models relationships between transaction positions, whereas the Channel-wise Feature Attention in MaRENet operates across feature channels using the mask defined in Eq. (7). Temporal dependencies across transaction positions are instead learned by the GRU/LSTM. Therefore, MaRENet does not construct a $d \times d$ temporal attention matrix.

E. Pipeline Overview

We begin with each transaction record (amount, merchant, timestamp, etc.) and apply our **transactional expansion** (Section III-B) to package it together with its d most recent predecessors. This produces a fixed-size history window of length $d+1$ for every new purchase, ensuring that short-term bursts and gradual shifts in spending both appear in the model's view. On top of that, we amplify the two components "day of the week" and "hour of day" by simple scalar multipliers, so that the network sees more prominently the weekly and daily periodicities which are known to matter in fraud patterns. Each per-transaction window is processed by the *channel-wise feature-attention* block, producing a sequence of vectors $\mathbf{X}^{\text{out}}$ from Eq. (8). The output sequence is then processed by a RNN (using either a GRU or an LSTM). Instead of looking at the whole window at once, the RNN updates its internal state step by step. Let $\mathbf{h}_t \in \mathbb{R}^H$ be the hidden state at time step t. We calculate it using the current input vector and the previous state:

$$\mathbf{h}_t = \text{RNN}(\mathbf{x}_t^{\text{out}}, \mathbf{h}_{t-1}) \in \mathbb{R}^H, \tag{9}$$

where $\mathbf{x}_t^{\text{out}} \in \mathbf{X}^{\text{out}}$ denotes the t-th row of $\mathbf{X}^{\text{out}}$, with $t = 0, 1, \ldots, d$, and when $t = 0$, the previous state $\mathbf{h}_{t-1}$ is set to a zero vector.

Finally, to predict if a transaction is fraud, we pass the summary vector $\mathbf{h}_d$ through a simple classifier and convert the output into a probability using the sigmoid function:

$$\hat{y} = \sigma(\ell) = \frac{1}{1 + e^{-\ell}}, \tag{10}$$

with $\ell = \mathbf{w}^\top \mathbf{h}_d + b$, where $\mathbf{w}$ and b are learnable parameters. This yields the predicted probability that the transaction sequence corresponds to fraudulent behavior.

MaRENet is trained using the Binary Cross-Entropy (BCE) objective:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_{i=1}^{N} \big[y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i)\big]. \tag{11}$$

This loss is the negative log-likelihood of a Bernoulli model and yields probabilistic outputs that are suitable for threshold-based decision making.

IV. Evaluation

A. Dataset

Our experiments use the *Credit Card Transactions Fraud Detection* dataset from Kaggle.[1] It has about 1.85 million credit card transactions from 999 unique users, and each record has 22 features that cover time, transaction details, and basic demographics. We call this main dataset **Dataset 1** and focus on it because its source is clear: the transactions were created with Brandon Harris's *Sparkov* framework, a transparent and checkable simulation tool for credit-card activity [24], [25]. Key statistics are summarized in Table I.

TABLE I
DATASET STATISTICS OF DATASET 1

Attribute	Value
Total transactions (n)	1,852,394
Number of users ($\lvert U \rvert$)	999
Attributes per transaction (m)	22
Fraudulent rate	0.52%

Besides Dataset 1, we also evaluate MaRENet on a second public dataset from Kaggle[2], referred to as **Dataset 2**. Dataset 1 is used as the primary benchmark because its data-generation process and feature definitions are clearly documented. Dataset 2 contains more transactions and users, providing an additional evaluation at a larger scale. However, its origin and data-generation process are less documented than those of Dataset 1. The Dataset 2 results are therefore treated as a supplementary large-scale evaluation rather than as strong evidence of generalization to real-world transaction streams. Key statistics are shown in Table II.

B. Implementation Detail

We set the history window length d using the heuristic described in Section III-B. For users with fewer than d past transactions (e.g., at the beginning of their timeline), we pad

[1] https://www.kaggle.com/datasets/kartik2112/fraud-detection

[2] https://www.kaggle.com/datasets/orogunadebola/credit-card-transaction-dataset-fraud-detection

TABLE II
DATASET STATISTICS OF DATASET 2

Attribute	Value
Total transactions (n)	8,580,255
Number of users ($\|U\|$)	9,968
Attributes per transaction	43
Fraudulent rate	1.10%

the window by repeating the earliest available transaction, so that every input window keeps the same length.

To avoid information leakage between transactions belonging to the same user, we use a user-disjoint split: 80% of users are assigned to the training set and the remaining 20% to the test set. The prediction target remains transaction-level fraud. The split is performed before preprocessing. All categorical encoders and numerical scalers are fitted using the training users only. Categorical feature values appearing in test transactions but not observed in the training data are assigned a reserved unknown value. The model is trained using the Adam optimizer. Table III summarizes the main training settings, including batch size, learning rate, learning rate decay, total epochs, and the chosen window length d.

TABLE III
MODEL HYPERPARAMETER SETTINGS

Hyperparameter	Value
Batch size	256
Optimizer	Adam
Initial learning rate	0.001
Learning rate decay epochs	50, 75
Total training epochs	100
History window length (d)	1854
Train/test split	80% / 20% (user-level)

C. Selected Baselines

Both classical ML models and recent DL models for credit card fraud detection are considered. The ML baselines include KNN, Logistic Regression, and Decision Tree, together with strong ensemble methods such as LightGBM, Random Forest, CatBoost, XGBoost, and Stacking or Voting classifiers. For reproducibility, we report the key hyperparameters used for each ML baseline in Table IV.

For DL models, RNN models such as GRU [20] and LSTM [21] are included as baselines. We also add two advanced sequential models: TH-LSTM [7], which uses time-aware gates and historical attention, and STAN [22], which combines spatio-temporal attention with 3D convolutional layers.

All models use the same user-level train/test split and preprocessing procedure. No resampling is applied to the training data. The recurrent models use standard binary cross-entropy without class weighting, and the results in Table V use a fixed decision threshold of 0.5 rather than test-set threshold tuning. Classical machine-learning baselines classify each transaction independently using only the feature vector of that transaction and do not use transactions from other time steps; they are therefore included as reference baselines. Sequence-based models use the current transaction together with preceding transactions associated with the same user to form a transaction history. The main architectural comparison focuses on GRU, LSTM, TH-LSTM, STAN, and MaRENet. In all cases, the prediction target is whether the current transaction is fraudulent, so the task is transaction-level fraud detection rather than account-level fraud detection.

TABLE IV
KEY HYPERPARAMETERS OF ML BASELINES

Model	Key hyperparameters
KNN	n_neighbors= 5
	$p = 2$
Logistic Regression	$C = 1.0$
	max_iter= 1000
Decision Tree	min_samples_split= 2
	min_samples_leaf= 1
Random Forest	n_estimators= 100
LightGBM	n_estimators= 100
	learning_rate= 0.1
	num_leaves= 31
XGBoost	n_estimators= 100
	max_depth= 6
	learning_rate= 0.3
CatBoost	iterations= 100
	depth= 6
	learning_rate= 0.1

D. Results

Table V summarizes the performance of all models on Dataset 1 and Dataset 2. **Bold** values mark the best result in each column, and underlined values mark the second-best result. Overall, MaRENet gives the strongest performance among the compared methods. On Dataset 1, the best tree-based baseline (XGBoost) reaches an F1-score of 0.9001, while the GRU and LSTM baselines achieve around 0.948 (0.9489 and 0.9484). In contrast, MaRENet reaches an F1-score of 0.9694, outperforming both classical machine learning models and stronger sequential baselines such as TH-LSTM (0.9413) and STAN (0.9029). On Dataset 2, MaRENet achieves the best F1-score of 0.9883, which is higher than the GRU/LSTM baselines (0.9666 and 0.9679) and recent models such as TH-LSTM (0.9690) and STAN (0.9443).

These results show consistent performance across the two evaluated datasets. The ablation results in Subsection IV-F indicate that this improvement is associated with the proposed modules: Temporal Feature Scaling adjusts the selected temporal features through learnable scaling factors, while Channel-wise Feature Attention applies a learnable mask across all feature channels before recurrent encoding. Together, these modules help the model capture temporal patterns and feature-level signals for transaction-level fraud detection.

E. Time-Ordered Evaluation

An additional experiment was conducted on Dataset 1 using a time-ordered split. All 1,852,394 transactions were sorted by timestamp and divided into 70% training, 10% validation, and 20% test sets. Transactions with the same timestamp were kept in the same split. Preprocessing was based on the training set only, and categorical feature values appearing in the validation or test sets but not observed in the training set

TABLE V
PERFORMANCE COMPARISON OF ALL MODELS ACROSS DATASET 1 AND DATASET 2

Model	**Dataset 1**				**Dataset 2**			
	Accuracy	Precision	Recall	F1-score	Accuracy	Precision	Recall	F1-score
KNN	0.9965	0.7559	0.4970	0.5997	0.9891	0.5073	0.5356	0.5211
Logistic Regression	0.9948	0.5013	0.3623	0.4206	0.9888	0.4927	0.3943	0.4380
Decision Tree	0.9965	0.7113	0.5485	0.6194	0.9865	0.3949	0.4094	0.4020
LightGBM	0.9978	0.8069	0.7570	0.7811	0.9902	0.5649	0.5039	0.5326
Random Forest	0.9988	0.9182	0.8383	0.8764	0.9904	0.5816	0.4657	0.5173
CatBoost	0.9987	0.9306	0.8082	0.8651	0.9900	0.5510	0.5132	0.5314
XGBoost	0.9990	0.9426	0.8612	0.9001	0.9901	0.5586	0.5099	0.5331
Stacking classifier	0.9989	0.9271	0.8619	0.8933	0.9903	0.5614	0.5416	0.5513
Voting classifier	0.9990	0.9276	0.8691	0.8974	0.9902	0.5614	0.5166	0.5381
GRU [20]	0.9995	0.9701	0.9286	0.9489	0.9993	0.9626	0.9706	0.9666
LSTM [21]	0.9995	**0.9782**	0.9241	0.9484	0.9994	0.9792	0.9569	0.9679
TH-LSTM [7]	0.9994	0.9522	0.9307	0.9413	0.9994	0.9854	0.9532	0.9690
STAN [22]	0.9990	0.9184	0.8879	0.9029	0.9989	0.9654	0.9242	0.9443
MaRENet-GRU	**0.9997**	0.9765	**0.9623**	**0.9694**	0.9995	0.9848	0.9652	0.9749
MaRENet-LSTM	0.9996	0.9730	0.9583	0.9656	**0.9997**	**0.9920**	**0.9846**	**0.9883**

TABLE VI
PERFORMANCE USING THE TIME-ORDERED SPLIT. RESULTS ARE REPORTED AS MEAN ± STANDARD DEVIATION OVER THREE RANDOM SEEDS

Model	Precision	Recall	F1	AUPRC	Recall@FPR$\leq 1\%$	ROC-AUC
GRU	0.9489 ± 0.0072	0.8930 ± 0.0154	0.9200 ± 0.0049	0.9457 ± 0.0039	0.9629 ± 0.0071	0.9951 ± 0.0012
LSTM	0.9420 ± 0.0105	0.9143 ± 0.0125	0.9279 ± 0.0087	0.9533 ± 0.0086	0.9701 ± 0.0068	0.9951 ± 0.0010
TH-LSTM	$\mathbf{0.9796 \pm 0.0116}$	0.9143 ± 0.0219	0.9456 ± 0.0083	0.9668 ± 0.0079	0.9800 ± 0.0053	0.9980 ± 0.0002
MaRENet-GRU	0.9548 ± 0.0031	$\mathbf{0.9382 \pm 0.0084}$	0.9464 ± 0.0040	0.9760 ± 0.0037	$\mathbf{0.9864 \pm 0.0041}$	$\mathbf{0.9987 \pm 0.0004}$
MaRENet-LSTM	0.9782 ± 0.0025	0.9320 ± 0.0115	$\mathbf{0.9546 \pm 0.0050}$	$\mathbf{0.9762 \pm 0.0027}$	0.9862 ± 0.0017	0.9986 ± 0.0005

TABLE VII
ABLATION STUDY ON MARENET-RNN BY REMOVING KEY COMPONENTS (DATASET 1 AND DATASET 2)

Variant	**Dataset 1**				**Dataset 2**			
	Acc	**Prec**	**Rec**	**F1**	**Acc**	**Prec**	**Rec**	**F1**
Our MaRENet (full model)	**0.9997**	**0.9765**	**0.9623**	**0.9694**	**0.9997**	**0.9920**	**0.9846**	**0.9883**
Without Channel-wise Feature Attention	0.9994	0.9662	0.9186	0.9418	0.9994	0.9838	0.9573	0.9704
Without Day/Hour scaling	0.9995	0.9744	0.9362	0.9549	0.9995	0.9790	0.9673	0.9731
Without both	0.9994	0.9388	0.9407	0.9398	0.9994	0.9825	0.9523	0.9672

were assigned a reserved unknown value. Transaction histories are carried across the split boundaries: validation transactions can use earlier transactions from the training period, while test transactions can use earlier transactions from the training and validation periods, as well as earlier test transactions. These histories are user-specific and strictly causal; future transactions and target labels are never used. The history length was calculated from the training set using the heuristic in Eq. (4), resulting in $d = 1319$. GRU, LSTM, TH-LSTM, MaRENet-GRU, and MaRENet-LSTM were evaluated with random seeds 42, 123, and 2026. The validation set was used for early stopping, model selection, and threshold selection.

This evaluation focuses on metrics that reflect fraud-class performance instead of relying on accuracy alone. In the Time-Ordered Evaluation, AUPRC and Recall@FPR$\leq 1\%$ are reported together with Precision, Recall, F1-score, and ROC-AUC. AUPRC measures performance across different precision–recall trade-offs, while Recall@FPR$\leq 1\%$ shows how much fraud can be detected when the false-positive rate is limited to 1%. Table VI shows that both MaRENet variants perform well on the time-ordered data. Compared with GRU, MaRENet-GRU improves F1 from 0.9200 to 0.9464 and AUPRC from 0.9457 to 0.9760. Compared with LSTM, MaRENet-LSTM improves F1 from 0.9279 to 0.9546 and AUPRC from 0.9533 to 0.9762. MaRENet-GRU achieves the highest Recall and Recall@FPR$\leq 1\%$, while MaRENet-LSTM achieves the highest F1-score and AUPRC. These results show that MaRENet remains effective for transaction-level fraud detection when trained on earlier transactions and evaluated on later transactions.

F. Ablation Study

Table VII reports an ablation study in which key components are removed from the full model on both datasets. When we remove the Channel-wise Feature Attention block, performance drops on both datasets, mainly in recall and F1. On Dataset 1, the F1 score decreases from 0.9694 to 0.9418, and recall drops from 0.9623 to 0.9186. On Dataset 2, the F1 score decreases from 0.9883 to 0.9704, and recall drops from 0.9846 to 0.9573. When we remove the Day/Hour scaling module (learnable weights for `day_of_week` and `trans_hour`), the F1 score also decreases on both datasets. On Dataset 1, F1 declines from 0.9694 to 0.9549, and on Dataset 2, it reduces from 0.9883 to 0.9731. If we remove both modules, the F1 score becomes 0.9398 on Dataset 1 and 0.9672 on Dataset 2. Overall, these results suggest that both feature attention and temporal scaling help improve fraud detection performance, especially by improving recall and F1.

TABLE VIII
EXTENSION OF TEMPORAL FEATURE SCALING ON DATASET 1

Variant	Temporal features	Precision	Recall	F1	AUPRC
Full MaRENet	Day/hour	0.9693	**0.9316**	**0.9500**	**0.9716**
Extended TFS-Month	Day/hour/month	**0.9823**	0.8942	0.9362	0.9679
Extended TFS-Weekend	Day/hour/weekend	0.9748	0.8834	0.9268	0.9645
Extended TFS-Combined	Day/hour/month/weekend	0.9414	0.8999	0.9202	0.9552

TABLE IX
ADDITIONAL ABLATION RESULTS ON DATASET 1 WITH $d = 500$ AND RANDOM SEED 42

Variant	Temporal features	Scaling	Norm	Residual	Precision	Recall	F1	AUPRC
Full MaRENet	Day/hour	Learnable	Yes	Yes	0.9693	**0.9316**	**0.9500**	0.9716
Fixed temporal scaling	Day/hour	Fixed	Yes	Yes	0.9823	0.9191	0.9497	**0.9729**
Without normalization	Day/hour	Learnable	No	Yes	**0.9834**	0.8901	0.9344	0.9685
Without residual	Day/hour	Learnable	Yes	No	0.9655	0.9295	0.9472	0.9659
Without normalization and residual	Day/hour	Learnable	No	No	0.9771	0.9057	0.9400	0.9706
Removed temporal features	Removed	None	Yes	Yes	0.9699	0.8507	0.9064	0.9565

We also tested Temporal Feature Scaling with additional temporal features: month, weekend, and both month and weekend. A shorter history length of $d = 500$ was used as a common setting for these controlled ablation experiments to reduce the computational cost of evaluating multiple variants. All variants use the same $d = 500$ and random seed 42 so that the comparison focuses on the effect of the temporal features. This setting is used only for the ablation analysis and does not replace the main Dataset 1 setting of $d = 1854$. For the extended settings, the learned effective scaling factors were $s_{\text{month}} = 0.0968$ for TFS-Month, $s_{\text{weekend}} = 0.0815$ for TFS-Weekend, and $s_{\text{month}} = 0.0572$ and $s_{\text{weekend}} = 0.2848$ for TFS-Combined, where $s_j = 1 + \tanh(\alpha_j)$.

Table VIII shows that the original day/hour setting achieves the highest F1-score and AUPRC among the tested variants. Adding month improves precision but reduces recall and F1-score, while adding weekend or combining month and weekend also gives lower overall performance. These results indicate that, among the temporal features tested on Dataset 1, adding month or weekend does not improve overall performance over the original day/hour setting. To examine the model components further, fixed and learnable Temporal Feature Scaling, removal of the temporal features, normalization, and the residual connection in Channel-wise Feature Attention are compared using the same GRU backbone and training protocol. All variants use Dataset 1 with $d = 500$ and random seed 42. Table IX shows that removing the temporal features causes the largest reduction in F1-score. Fixed and learnable Temporal Feature Scaling give similar performance, indicating that learnable scaling alone is not responsible for the improvement. Removing normalization also reduces F1-score, while removing the residual connection has a smaller effect.

Threshold sensitivity was further examined for Full MaRENet-GRU using the same setting with $d = 500$ and random seed 42. Figure 5 shows that Precision increases as the threshold increases, while Recall decreases. Around the predefined threshold of 0.5, the F1-score remains relatively stable, with values of 0.9506, 0.9500, and 0.9502 at thresholds 0.45, 0.50, and 0.55, respectively. The threshold sweep is used only for sensitivity analysis, and the ablation results continue to use the fixed threshold of 0.5.

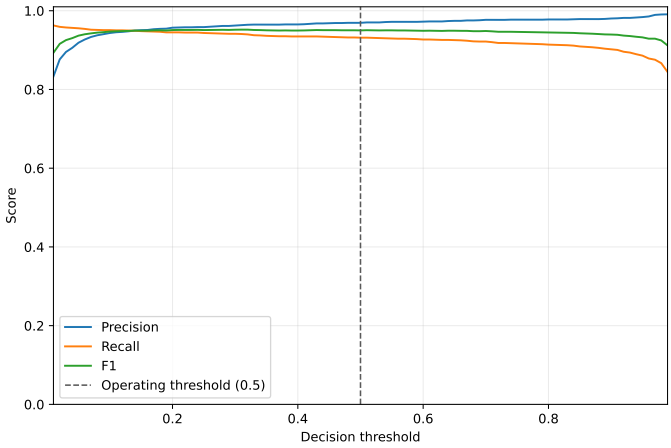


Fig. 5. Sensitivity of Precision, Recall, and F1-score to the decision threshold for Full MaRENet-GRU on Dataset 1 with $d = 500$ and random seed 42. The dashed line indicates the predefined operating threshold of 0.5.

TABLE X
EFFECT OF MEMORY LENGTH d ON DETECTION PERFORMANCE

Memory Length	F1-score
50	0.9568
200	0.9600
500	0.9627
1854 (heuristic)	**0.9694**
2000	0.9608

(a) Dataset 1

Memory Length	F1-score
50	0.9738
200	0.9771
500	0.9819
860 (heuristic)	**0.9883**
1000	0.9864

(b) Dataset 2

G. Computational Efficiency Analysis

We also study how the transactional memory length d affects performance on both Dataset 1 and Dataset 2. Although Eq. (4) provides a heuristic value based on dataset statistics, this choice may not fit all deployment settings. We therefore test several fixed memory lengths to understand the tradeoff between detection performance and computational cost. As shown in Table X, increasing d generally improves the F_1 score because the model can use more behavioral context. On Dataset 1, the F_1 score increases from 0.9568 at $d = 50$

TABLE XI
PERFORMANCE AND INFERENCE EFFICIENCY OF MARENET-GRU AND TH-LSTM AT $d = 1854$ ON DATASET 1

Model	F1-score	Parameters	Median latency (ms)	Throughput (transactions/s)	Peak inference memory (MiB)
MaRENet-GRU	**0.9694**	**52,820**	**2.87**	**12,164.14**	664.21
TH-LSTM	0.9413	143,745	1,461.45	82.78	**30.36**

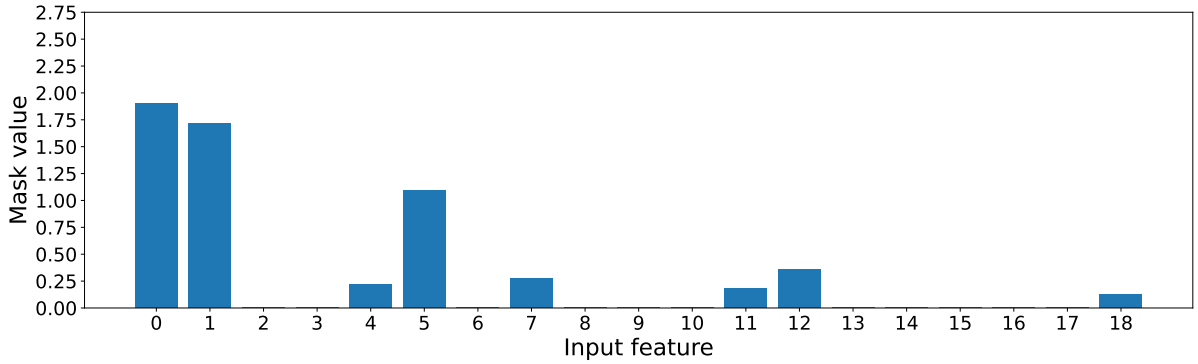


Fig. 6. Learned feature-mask values across input channels.

to 0.9627 at $d = 500$, and reaches the best value of 0.9694 at the heuristic length $d = 1854$. However, when we further increase the memory length to $d = 2000$, the F_1 score drops to 0.9608, suggesting that an overly long history may add more noise than useful information. A similar trend is observed on Dataset 2. The F_1 score improves from 0.9738 at $d = 50$ to 0.9819 at $d = 500$, and peaks at 0.9883 with the heuristic length $d = 860$. When d increases to 1000, the F_1 score slightly decreases to 0.9864. Overall, the gains become smaller after $d = 500$ on both datasets, while the computation cost keeps increasing roughly linearly with d. For computationally constrained settings, smaller values such as $d = 200$ or $d = 500$ may provide a better balance between cost and accuracy, even though the heuristic setting achieves the best F_1 in our experiments.

Table XI compares MaRENet-GRU and TH-LSTM at the same history length $d = 1854$ on Dataset 1 using a single NVIDIA RTX 3060 GPU. Latency was measured with batch size 1 after 50 warm-up iterations over 500 timed runs, while throughput was measured with batch size 128 after 20 warm-up batches over 200 timed batches. CUDA synchronization was applied before and after each timed measurement, and latency and throughput were measured using CUDA FP16 automatic mixed precision. Peak inference memory was measured separately with batch size 128. Table XI shows that MaRENet-GRU achieves a higher F1-score with 63.3% fewer trainable parameters than TH-LSTM. Under the reported benchmark conditions, MaRENet-GRU achieves approximately $509\times$ lower latency and $147\times$ higher throughput. MaRENet-GRU uses more inference memory, but its peak memory remains below 0.7 GiB. These results show that MaRENet-GRU is more efficient than TH-LSTM in this benchmark.

a) Interpreting Feature Contributions via Learned Mask: MaRENet also provides an interpretable view of feature-level relevance. The channel-wise attention module learns a feature mask during training, as described in Section III and Eq. (7). As shown in Fig. 6, relatively high mask values are assigned to several informative attributes, including transaction category, transaction amount, transaction hour, nighttime flag, customer age group, merchant ID, and gender, corresponding to channel indices 0, 1, 4, 5, 11, 12, and 18, respectively.

These learned values indicate that MaRENet places substantial emphasis on transaction amount and temporal information, together with product category, merchant identity, and selected demographic attributes. This channel-wise prioritization improves the transparency of the learned representation by indicating which input features receive greater emphasis, without requiring an additional post-hoc explanation module.

V. CONCLUSION

We proposed MaRENet, a credit card fraud detection framework that combines Transactional Expansion, Temporal Feature Scaling, Channel-wise Feature Attention, and a GRU/LSTM recurrent encoder. Experiments on two large-scale public transaction datasets show strong fraud detection performance. Additional experiments with a time-ordered split show that MaRENet remains effective when trained on earlier transactions and tested on later ones. Under the reported benchmark conditions at $d = 1854$, MaRENet-GRU achieves lower latency and higher throughput than TH-LSTM while using fewer trainable parameters. However, the reported efficiency measurements do not by themselves establish readiness for real-time deployment, which will be further evaluated in future work.

However, some limitations should be acknowledged. MaRENet has not yet been evaluated on real-world transaction data, which limits our ability to assess its generalizability. In addition, the model uses a fixed history length d for all users, despite differences in their transaction frequencies. Future work will focus on evaluating MaRENet on real-world datasets, exploring adaptive history lengths, and incorporating spatial and device-related features.

DECLARATIONS

Competing interests The authors declare no competing interests.

Ethics approval Not Applicable.

Consent to participate Not Applicable.

Consent to publish Not Applicable.

Funding Not Applicable.

REFERENCES

[1] G. Gianini, L. G. Fossi, C. Mio, O. Caelen, L. Brunie, and E. Damiani, "Managing a pool of rules for credit card fraud detection by a game theory based approach," *Future Gener. Comput. Syst.*, vol. 102, pp. 549–561, 2020.

[2] Y. Wu, Y. Xu, and J. Li, "Feature construction for fraudulent credit card cash-out detection," *Decis. Support Syst.*, vol. 127, 2019.

[3] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: a realistic modeling and a novel learning strategy," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 8, pp. 3784–3797, 2018.

[4] S. Bhattacharyya, S. Jha, K. Tharakunnel, and J. Westland, "Data mining for credit card fraud: A comparative study," *Decis. Support Syst.*, vol. 50, no. 3, pp. 602–613, 2011.

[5] E. W. T. Ngai, Y. Hu, Y. H. Wong, Y. Chen, and X. Sun, "The application of data mining techniques in financial fraud detection: A classification framework and an academic review," *Decis. Support Syst.*, vol. 50, no. 3, pp. 559–569, 2011.

[6] A. Dal Pozzolo, O. Caelen, Y.-A. Le Borgne, S. Waterschoot, and G. Bontempi, "Adaptive machine learning for credit card fraud detection," *Future Gener. Comput. Syst.*, vol. 86, pp. 548–555, 2018.

[7] Y. Xie, G. Liu, C. Yan, C. Jiang, M. Zhou, and M. Li, "Learning transactional behavioral representations for credit card fraud detection," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 4, pp. 5735–5748, Apr. 2024.

[8] Y. Xie, G. Liu, M. Zhou, L. Wei, H. Zhu, R. Zhou, and L. Cao, "A spatial–temporal gated network for credit card fraud detection by learning transactional representations," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 4, pp. 6978–6989, 2024.

[9] H. Zhu, M. Zhou, Y. Xie, and A. Albeshri, "A self-adapting and efficient dandelion algorithm and its application to feature selection for credit card fraud detection," *IEEE/CAA J. Autom. Sinica*, vol. 11, no. 2, pp. 377–390, 2024.

[10] K. Cheng, Y. Zhang, X. He, W. Chen, C. Li, J. Zhou, and J. Cheng, "Skeleton-based action recognition with shift graph convolutional network," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020, pp. 183–192.

[11] A. N. Upadhyaya, S. K. Mittal, J. Kotecha, and S. V. Ramana, "Enhancing credit card fraud detection with K-nearest neighbours (KNN): A machine learning approach," *J. Inf. Syst. Eng. Manag.*, vol. 10, no. 2, 2025.

[12] T. Wang and Y. Zhao, "Credit card fraud detection using logistic regression," in *Proc. Int. Conf. Big Data, Inf. Comput. Netw. (BDICN)*, 2022.

[13] E. Flondor, L. Donath, and M. Neamtu, "Automatic card fraud detection based on decision tree algorithm," *Appl. Artif. Intell.*, vol. 38, no. 1, 2024.

[14] A. H. M. Aburbeian and H. I. Ashqar, "Credit card fraud detection using enhanced random forest classifier for imbalanced data," in *Proc. Int. Conf. Adv. Comput. Res. (ACR'23)*, ser. Lecture Notes in Networks and Systems, vol. 700, 2023, pp. 605–616.

[15] A. A. Taha and S. J. Malebary, "An intelligent approach to credit card fraud detection using an optimized light gradient boosting machine," *IEEE Access*, vol. 8, pp. 25 579–25 587, 2020.

[16] A. Purwar and Manju, "Credit card fraud detection using XGBoost for imbalanced data set," in *Proc. Int. Conf. Adv. Comput. Data Sci.*, 2023.

[17] V. Suryanarayana *et al.*, "An efficient implementation of credit card fraud detection using catboost algorithm," *Indones. J. Electr. Eng. Comput. Sci.*, vol. 38, no. 3, pp. 1914–1923, 2025.

[18] E. B. Abdelghafour *et al.*, "Enhancing credit card fraud detection using a stacking model approach and hyperparameter optimization," *Int. J. Adv. Comput. Sci. Appl. (IJACSA)*, vol. 15, no. 10, 2024.

[19] M. A. Mim, N. Majadi, and P. Mazumder, "A soft voting ensemble learning approach for credit card fraud detection," *Heliyon*, vol. 10, no. 3, 2024.

[20] V. Suganthi and J. Jebathangam, "A novel approach for credit card fraud detection using gated recurrent unit (GRU) networks," *J. Intell. Fuzzy Syst.*, pp. 1606–1610, 2024.

[21] J. Jurgovsky *et al.*, "Sequence classification for credit-card fraud detection," *Expert Syst. Appl.*, vol. 100, pp. 234–245, 2018.

[22] D. Cheng, S. Xiang, C. Shang, Y. Zhang, F. Yang, and L. Zhang, "Spatio-temporal attention-based neural network for credit card fraud detection," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 01, 2020, pp. 362–369.

[23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.

[24] B. Harris, "Sparkov: Markov-chain based fraud detection system in spark," 2022, gitHub repository. [Online]. Available: https://github.com/namebrandon/Sparkov

[25] ——, "Sparkov data generation: Synthetic credit card transaction generator," 2022, gitHub repository. [Online]. Available: https://github.com/namebrandon/Sparkov_Data_Generation